SOFTWARE CORRECTNESS / EXACT ALGEBRA

Radical Denesting After the First Repair

A second audit of `StradFixed.wl` and a certified, budgeted redesign

Prepared for Vladimir Reshetnikov

5 September 2026

Abstract

The corrected denester in the *RadicalDenest* repository is substantially safer than its predecessor: in particular, its multiplier engine checks proposed answers against the original target, rather than a moving intermediate target. Nevertheless, its advertised guarantees are broader than the implementation supports. A user-supplied solver can replace a numerical radical inside a symbolic host without local certification; effective option defaults are not consistently propagated; expensive algebraic operations and combinatorial generation escape the intended search budget; and the multiplier-cap calculation still admits oversized batches. A separate failure-path defect arises because a failed minimal-polynomial computation can be mistaken for a constant polynomial by a public rationalization helper.

This article separates demonstrated source-level consequences, mathematical counterexamples, proof gaps, and remaining engineering risks. It derives the algebra behind the multiplier search and several useful exact fast paths, corrects two claims in the earlier improvement plan, and presents a proposed Wolfram Language replacement with local equality gates, explicit resource boundaries, a finite multiplier queue, and structured diagnostics. The accompanying package includes native regression tests and independently executed exact mathematical checks. Neither unrestricted completeness nor native-tested production readiness is claimed.

Pinned baseline. `VladimirReshetnikov/RadicalDenest`, commit `6d7739bda3478f0d9cd70ffb9f7c49ee2576bee7`.

Execution status. The native Wolfram tests were *not run*: the connected evaluator returned an MCP HTTP 404, and no local kernel was present. The independent Python/SymPy and lexical checks were run and passed. Their scope is stated precisely in Section 8.

Contents

1 Executive assessment and scope

The central recommendation is to retain the repaired program's most important principle—candidate generation is heuristic, but acceptance is exact—and make that principle structural rather than dependent on which public entry point or wrapper branch happens to execute.

The current code should not be described simply as mathematically unsound. Its default engine has an immutable `problem`, tests candidates against it, and rechecks its selected result. These are real repairs, and they eliminate the earlier target-drift failure. The remaining critical issue is that this protection does not surround every path through the public interface. In particular, the custom-solver path for a symbolic host bypasses the local gate that would make the wrapper's guarantee compositional. Separately, the advertised budgets are largely scheduling checks, not boundaries around the work they purport to limit. Both distinctions matter when deciding what to fix first.[1, 2]

1.1 What was inspected

The review uses the pinned `src/corrected/StradFixed.wl`, the unified report under `docs/report`, and the earlier unified review, especially its corrected-version and future-improvement sections. Relevant literature was read in the repository's corrected editions: Borodin–Fagin–Hopcroft–Tompa (BFHT), Landau, Jeffrey–Rich, and Gkioulekas. The report's Honsbeek and cubic-in-quadratic developments were also used. The author's posted Gkioulekas PDF was checked independently, including a visual inspection of its indirect-denesting theorem. Official Wolfram documentation was consulted for the exact-algebraic zero test, algebraic canonicalization, polynomial coefficient fields, option lookup, and time constraints.[4, 3, 6, 7, 8, 9, 10, 11, 12, 13, 14]

There is an important provenance distinction. The files in `retypeset-2026` contain editorial corrections and qualifications; they are not publisher facsimiles or evidence that the original authors approved the changes. This article attributes historical results to the original literature, identifies repository-specific claims as such, and supplies fresh proofs for the local formulas actually used by the proposed implementation. It does not purport to audit every item in the literature directory.[5]

1.2 Evidence levels

Four labels are useful throughout this review.

Source-level consequence.
A control-flow or evaluation consequence follows from the displayed code and documented language semantics. A small native probe is supplied where useful; the result is not represented as an observation from an executed Wolfram session.

Exact mathematical counterexample.
A stated algebraic or counting claim is disproved by a concrete construction. These examples do not depend on the denester being executed.

Proof gap.
The stated guard does not justify the transformation performed. This is not automatically evidence that the relevant internal state is reachable through the current normalizer.

Engineering limitation.
The implementation has a restricted search space, a costly representation, or an interface ambiguity. Such a limitation is not a wrong mathematical answer by itself.

The accompanying `StradImproved.wl` is a proposed redesign. It has undergone manual review, mathematical cross-checks, and limited lexical checking, but not native parsing/evaluation or performance testing. That distinction is especially important for Wolfram Language, where pattern specificity, evaluation attributes, option scoping, and algebraic-number representations are part of program behavior.

1.3 Priority map

ID	Priority	Finding	Evidence
F01	Critical	A custom solver is not locally certified inside a symbolic host.	Source-level consequence.
F02	High	Effective defaults of aliases/wrappers do not reliably reach the engine.	Source-level consequence.
F03	High	Time and trial budgets do not cover the whole advertised operation.	Source-level consequence.
F04	High	Multiplier batches exceed the cap; truncation follows exponential generation.	Exact counting examples.
F05	High	Invalid limits are not rejected; an infinite cap can make a counting loop nonterminating.	Source-level consequence.
F06	High	A failed minimal polynomial can pass a polynomial test and corrupt a public helper's result.	Conditional failure-path analysis.
F07	High	Whole-host simplification can use ambient assumptions not stated by the interface.	Contract and source analysis.
F08	Medium	Certification has unbounded preliminaries, an unrigorous numeric rejection, and a narrow fallback path.	Source analysis; no false-acceptance claim.
F09	Medium	A positive aggregate factor is not a license for termwise fractional-power extraction.	Proof gap plus mathematical example.
F10	Medium	Polished candidates are not individually refiltered at every internal choice.	Search-quality risk; final numeric gate remains.
F11	Medium	The cost is inconsistent with its documentation and is not compositional.	Representation analysis.
F12	Medium	Degree progress, syntactic deduplication, and growing positional state limit search quality.	Engineering limitation.
F13	Medium	Root conversion is all-or-nothing; general expression traversal has an underspecified domain.	Coverage and interface analysis.
F14	Plan repair	The earlier fourth-root criterion has the wrong sign; norm divisors alone are not a sufficient multiplier family.	Exact mathematical counterexamples.

2 The mathematical and software contract

2.1 Four questions that must not be conflated

Given an exact algebraic value α, the following are different tasks:

(i) decide whether another expression denotes the same selected complex number;

(ii) reconstruct that number in a requested output language;

(iii) find a representation of lower radical depth;

(iv) prove that no representation of smaller allowed depth exists.

The multiplier engine addresses the third task by searching and uses the first task to protect its output. It does not solve the fourth task merely because the search ends. Likewise, converting everything to `Root` can solve a canonical representation problem without solving radical denesting. The unified report correctly emphasizes these distinctions, as does the general literature when its base fields and representation models are made explicit.[4, 7, 8]

For principal powers, define, for $z \neq 0$,

$$z^r = \exp(r \operatorname{Log} z), \qquad \operatorname{Log} z = \log|z| + i \operatorname{Arg} z, \quad -\pi < \operatorname{Arg} z \leq \pi.$$

An odd real root and a principal fractional power of a negative real number are not the same operation. For example,

$$\operatorname{realroot}_3(-8) = -2, \qquad (-8)^{1/3} = 1 + i\sqrt{3}.$$

The reviewed code works with Wolfram `Power`; the proposed replacement retains that convention. It does not silently reinterpret a fractional power as a real odd root.

2.2 A useful precise acceptance contract

For a numerical input E in the supported grammar, a successful simplifier should return either E unchanged or an expression R satisfying

$$\operatorname{value}(R) = \operatorname{value}(E), \qquad C(R) <_{\text{lex}} C(E). \tag{1}$$

The equality must refer to the original selected value, not to a polynomial with the same roots or to an equation satisfied after taking a power. The cost is an explicit syntactic objective, not a claim about minimum radical depth in every mathematically possible expression language.

For a symbolic host, a different formulation is needed. Suppose an inert mathematical expression contains an exact numerical subtree A. Replacing that subtree by B is justified by $A = B$, with no assumption on the surrounding variables. It is not necessary to prove that the entire host is an algebraic number; it is necessary to certify *each replaced numerical subtree.* This is the local contract violated by F01.

Proposition 2.1 (Local certificates compose)**.** *Let an expression be built from exact numerical leaves, symbolic leaves, lists, and arithmetic operations with principal-power semantics. Replace finitely many numerical subexpressions by equal selected numerical values, without otherwise altering the symbolic host. Wherever the original expression is defined, the resulting expression has the same value.*

Proof. Proceed by structural induction. Equal numerical leaves have the same value by hypothesis; unchanged symbolic leaves are identical. Addition, multiplication, and list formation depend only on the values of their arguments. A principal power likewise depends on the same ordered pair consisting of its base value and exponent value, so replacing an argument by an equal value does not alter branch selection. The same reasoning applies to division represented by a negative integer power, with its nonzero-denominator condition unchanged. Induction through the expression tree proves the claim. □

This is a statement about mathematical expressions, not arbitrary executable Wolfram programs. A held syntax tree, an assignment, an effectful user function, or a head that inspects the literal syntax of an argument is not covered by the proof. A software interface must either restrict its traversal domain or give a separate operational account of such constructs.

2.3 What the first repair already gets right

The current engine's candidate filter compares a proposed answer with `problem`, not with the latest multiplier-dependent expression. Its final numerical result is checked again. Its custom power-merging rule includes a meaningful branch guard. Its package/private contexts avoid the earlier global-symbol hazards, and diagnostics use an option instead of modifying `Print`. It also includes a direct quadratic-surd fast path and explicit cost comparisons. These are substantive improvements, not cosmetic edits.[1, 2]

Two tempting criticisms would therefore be wrong. First, the original-target drift should not be reported as still present in this revision. Second, the occurrence of `PossibleZeroQ` is not evidence of a heuristic acceptance oracle: the code specifies `Method -> "ExactAlgebraics"`, which the official documentation describes as guaranteed for explicit algebraic numbers. The actual issues are its surrounding domain/time checks and how the wrapper uses the predicate.[11]

3 Reconstructing the multiplier engine

3.1 The algebra behind a trial

Let $\alpha \neq 0$ be the immutable target, choose an integer $q \geq 2$, and put

$$\rho = \alpha^q.$$

For a nonzero exact algebraic multiplier m, let

$$\lambda = m^{1/q}, \qquad t = (m\rho)^{1/q}, \qquad p(X) = \operatorname{minpoly}_{\mathbb{Q}}(t).$$

Both fractional powers are principal. They need not satisfy $t = \lambda\alpha$. The discrepancy is a root of unity, which is precisely why branch recovery is an essential part of the algorithm.

The code computes a polynomial greatest common divisor with $X^q - m\rho$ over an algebraic coefficient field. A useful explicit choice is

$$K = \mathbb{Q}(m\rho), \qquad g(X) = \gcd_{K[X]}\big(p(X), X^q - m\rho\big).$$

The revised implementation represents the coefficient $m\rho$ by a single exact canonical algebraic element and requests that extension explicitly. The baseline's `Extension -> Automatic` is not inherently incorrect: Wolfram documents it as adjoining algebraic numbers appearing in the polynomials. The explicit generator is a representation and reproducibility improvement, not a correction to a universally invalid option.[12]

Theorem 3.1 (Branch-complete recovery from any trial root)**.** *If z is any root of $X^q - m\rho$ and $\alpha \neq 0$, then*

$$\alpha \in \left\{ \frac{z}{\lambda}\zeta_q^k : 0 \leq k < q \right\}, \qquad \zeta_q = e^{2\pi i/q}.$$

Proof. Since $\lambda^q = m$, we have

$$\left(\frac{z}{\lambda}\right)^q = \frac{m\rho}{m} = \rho = \alpha^q.$$

Thus $z/(\lambda\alpha)$ is a qth root of unity. Multiplication by the complete finite group of such roots produces α. This proof does not assume multiplicativity of principal qth roots. If $\alpha = 0$, every relevant z is zero and recovery is trivial; the denester normally encounters that value after ordinary simplification rather than in this search. □

The role of g is not to establish equality by itself. It is to expose roots through a polynomial of hopefully smaller degree, from which less nested expressions may be reconstructed. Every reconstructed orbit element must still be compared with the original α.

3.2 A completely worked trial

Take

$$\rho = 2 + \sqrt{3}, \qquad \alpha = \sqrt{2 + \sqrt{3}}, \qquad q = 2, \quad m = 2.$$

Then $t = 1 + \sqrt{3}$, so

$$p(X) = X^2 - 2X - 2.$$

Over $K = \mathbb{Q}(\sqrt{3})$,

$$\gcd\big(X^2 - 2X - 2, X^2 - 4 - 2\sqrt{3}\big) = X - 1 - \sqrt{3}.$$

Recovering the two orbit members gives

$$\pm\frac{1 + \sqrt{3}}{\sqrt{2}} = \pm\frac{\sqrt{2} + \sqrt{6}}{2}.$$

Only the positive choice equals the principal square root. Both satisfy the squared equation. The exact equality gate is therefore doing real mathematical work even in this elementary example.

The independent verification script also checks the trial

$$\rho = 5 + 2\sqrt{6}, \quad m = 2, \quad p(X) = X^2 - 4X - 2, \quad g(X) = X - 2 - \sqrt{6},$$

and both signs of the original target. These are small exact algebraic checks, not performance measurements of the Wolfram program.

3.3 Where the heuristic begins

The baseline chooses q by an LCM of exponent denominators at maximum syntactic depth, guesses initial multipliers from the expanded radicand, measures degrees, and uses discriminants to form further batches. It maintains a history and a mutable stack of partially explored batches. These are candidate-generation decisions; none is a completeness theorem.[1]

In particular, choosing an LCM does not prove that α^q has lower radical depth. For an individual outer rational power it is a natural way to remove that power, but for a sum or product there may be cross terms and nested radicals after exponentiation. The multiplier identity remains valid for any integer q, so this does not invalidate accepted answers. It can make a supposedly promising search step much more expensive than expected.

Likewise, algebraic degree and radical depth are different quantities. The two expressions

$$\sqrt{5 + 2\sqrt{6}} \quad \text{and} \quad \sqrt{2} + \sqrt{3}$$

denote the same degree-four number. Conversely, a depth-one expression such as $2^{1/2^k}$ can have degree 2^k. Degree is a useful scheduling feature, but it is neither the denesting objective nor a complete measure of computational cost. The literature's bounds must be interpreted with their input encodings and extension-field representations left visible.[4, 7, 8]

4 High-priority correctness and interface findings

4.1 F01: A custom solver bypasses local certification in a symbolic host

Location: `DenestRadicals`, marker resolution and final candidate selection. **Classification:** source-level consequence; the native diagnostic is supplied but was not executed.

The wrapper resolves a marker by calling its selected solver. With the default solver, the engine normally returns a certified numerical value. With a user-supplied solver, the wrapper relies on that callback's answer without adding a local equality check. The wrapper subsequently certifies the whole result only when the entire original expression is an exact algebraic number. A symbolic host fails that predicate and skips the whole-result gate.[1]

A minimal probe is

```
RadicalDenest`Strad[
  x + Sqrt[5 + 2 Sqrt[6]],
  "Solver" -> (0 &), "Factor" -> False
]
```

For an unassigned symbol x, the numerical radical can be replaced by zero. The resulting x is cheaper, and the original expression is not a numerical algebraic value, so the wrapper has no certification step capable of rejecting this change. This is a counterexample to the advertised claim that every replaced numerical subexpression is independently certified. The issue does not require the callback to be deliberately hostile: returning a sentinel, an approximate guess, or the wrong conjugate is enough.

Repair. Treat every solver as an untrusted proposal source. The marker or traversal node must retain its numerical target A and accept a callback answer B only after $B = A$ is certified. Apply the same rule to normalization, factoring, and polishing. The revised `choose[target,best,candidate,method]` is the shared commit point for such proposals. No callback result receives a privileged route around it.

4.2 F02: Effective option defaults are lost at two forwarding boundaries

Location: `Strad` aliases and `coreOpts` construction. **Classification:** source-level consequence.

The alias forwards explicit `opts` without looking up its own effective defaults. As a result, changing `Options[Strad]` does not necessarily change the behavior of a call that supplies no explicit rule. The inner wrapper similarly constructs core options from

```
FilterRules[{opts}, Options[DenestCore]]
```

which extracts explicit rules, not the effective values obtained from `Options[DenestRadicals]`. A changed wrapper default for `"MaxTrials"`, for example, may never reach the engine. Some values are dynamically scoped by the wrapper, which makes the behavior especially confusing: different options can appear to propagate through different mechanisms.[1]

Repair. Resolve the public entry point's effective options exactly once. Wolfram's explicit-head `OptionValue` form provides the appropriate semantics; in particular, explicit rules and the head's defaults have a defined precedence. Then pass one immutable configuration association through the session.[13]

The replacement deliberately gives each public entry point its own effective defaults. Setting `Strad`'s options affects `Strad`; setting `DenestRadicals`' options affects that head. The two default lists are initially

equal but are not treated as dynamically linked global configuration. This choice is documented rather than left to accidental forwarding behavior.

4.3 F03: The budgets are scheduling checks, not whole-call limits

Location: core initialization, the trial loop, `try`, generation routines, and recursive wrapper calls. **Classification:** source-level consequence.

The baseline checks its deadline and trial counter before a trial. A trial can then run `MinimalPolynomial`, `PolynomialGCD`, `Solve`, candidate enumeration, and polishing without a time boundary around the whole operation. Discriminants, integer factorization, and combinatorial batch generation can also occur between these checks. An operation that runs for a long time is not interrupted merely because the next loop iteration would notice an expired deadline.[1]

The same problem appears outside the trial loop. Exact-algebraic predicates, expansion, denominator rationalization, fast-path preprocessing, and some sign decisions can occur before the first trial check. Recursive passes and list handling can start fresh core sessions, resetting counters and deadlines rather than sharing the user's top-level budget. Therefore neither a zero trial limit nor a nominal time budget means what a reader might infer from the earlier guarantee. A zero trial limit can reasonably permit cheap formula evaluation, but that semantic choice must be explicit; a time budget needs a different implementation.

Repair. Place a top-level cooperative time and memory boundary around the work, including cost computation and traversal, and use a shared deadline and trial counter throughout all list elements and passes. Put separate boundaries around costly algebraic operations, taking the minimum of their local allocation and the remaining session time. Preserve the last *completed* certified whole-expression result for timeout fallback.

There remains an unavoidable software qualification: `TimeConstrained` is a kernel evaluation mechanism with granularity and interruption behavior, not a hard real-time process supervisor. Input evaluation before the call, option resolution, and small final report assembly are outside the proposed timed body. A callback using protected abort behavior is not sandboxed by this design. These limits should be stated instead of replacing one overstated time guarantee by another.[14]

4.4 F04: The multiplier cap still has two arithmetic leaks and one allocation leak

Location: `newMultipliers` and the post-first-pass product batch. **Classification:** exact counting counterexamples and source-level allocation analysis.

Suppose q is the root order and c is the intended cap. The code first finds t satisfying

$$q^t - 1 \le c,$$

but then forces at least one prime to be retained. For $q = 3$, $c = 1$, and the single prime 5, that produces the nontrivial divisors of 5^{q-1},

$$\{5, 25\},$$

so a cap of one admits two multipliers. The example can be generated from the polynomial $X^2 - X - 1$, whose discriminant is 5.

A second leak is independent of the forced minimum. After selecting a reduced prime subset, the routine unions the generated divisors with the original prime list. For $q = 2$, $c = 1$, and discriminant $65 = 5 \cdot 13$, the retained one-prime divisor family is $\{1, 5\}$, but union with the original primes and deletion of 1 yields

$$\{5, 13\}.$$

The polynomial $X^2 - X - 16$ has exactly this discriminant. Thus a prime discarded by the cap calculation can be reintroduced immediately afterward. These are not floating-point or rounding issues; they survive an exact integer calculation.

Finally, the later product batch uses a distributed product construction and truncates the completed list. If k promising factors each contribute a choice of exponent zero or one, the intermediate family has 2^k members before truncation. A small final list is not a bound on the time or memory used to obtain it. The same distinction applies to building a full root-of-unity orbit before filtering.[1]

Repair. Enforce capacity at a single queue insertion point, counting all admitted seeds and generated candidates for the numerical target. Bound proposal attempts as well, so an arbitrarily long stream of invalid or duplicate proposals cannot consume unbounded work. Generate exponent vectors and root-of-unity candidates incrementally. The replacement uses a finite FIFO queue, a per-target admission cap, a four-times-cap proposal bound, and bounded prefixes of mixed-radix exponent vectors. It never constructs the full divisor or subset-product family.

4.5 F05: Malformed or infinite limits can defeat termination

Location: option consumption throughout the search. **Classification:** source-level consequence.

The baseline does not comprehensively validate its option values. In the cap-counting loop, setting `"MultiplierCap" -> Infinity` makes every finite inequality $q^{t+1} - 1 \leq \infty$ true. The loop intended to compute the allowable number of primes therefore has no finite stopping point. Noninteger or negative capacities can produce invalid list-taking specifications or misleading fallback behavior; symbolic trial limits can turn loop guards into unevaluated conditions.

Repair. Validate before entering the session. Counts must be finite integers in their stated ranges, time allocations must be finite nonnegative numbers, and Boolean options must be actual `True` or `False`. Reject unknown keys and invalid values with a structured `Failure`. An infinite search can be a separate explicit mode in a future API, but it cannot be obtained safely by feeding `Infinity` into arithmetic derived for finite caps.

4.6 F06: A failed minimal polynomial can masquerade as a constant polynomial

Location: `RationalizeDenominator`; analogous validation weaknesses affect internal polynomial checks. **Classification:** conditional failure-path defect, not a reproduced ordinary-input failure.

The public helper contains the following sequence:

```
minpoly = Quiet[Check[MinimalPolynomial[denom, x], $Failed]];
If[! PolynomialQ[minpoly, x], Return[expr]];
q = PolynomialQuotient[minpoly, x - denom, x] /. x -> 0;
Expand[num*q]/(-(minpoly /. x -> 0))
```

The intended guard is insufficient. A symbol that is independent of x is a constant polynomial in x. Consequently `PolynomialQ[$Failed,x]` is not an assertion that a polynomial computation succeeded. If the preceding `Check` substitutes `$Failed`, the quotient of that constant by a linear polynomial is zero, and the helper can simplify its purported rationalization to zero. Unlike the final numerical engine result, this exported helper has no unconditional final equality gate.[1]

The precise claim is conditional: *when a minimal-polynomial message is caught and converted into the sentinel, the following validation does not contain the failure.* This review does not claim to have found a small normal

algebraic denominator that triggers that message in a native kernel. The supplied diagnostic separately prints the sentinel polynomial predicate, and the new tests require the replacement's stronger predicate to reject it.

Repair. Reject sentinels before mathematical predicates; require positive bounded degree; validate coefficients where a rational minimal polynomial is expected; and require a nonzero constant term for inversion. Better still, reconstruct an inverse from the polynomial and certify the local identity $d\,d^{-1} = 1$ before modifying the original expression. Section 6.6 derives that construction.

4.7 F07: Whole-host simplification leaks ambient assumptions

Location: final `safeSimplify[result]` selection. **Classification:** interface-contract and source-level issue.

The wrapper offers a global `Simplify` variant even for a symbolic host, and `safeSimplify` does not supply an explicit assumption policy. In a surrounding context with `$Assumptions = x > 0`, an expression containing $\sqrt{x^2}$ can be simplified to x. Such a transformation is valid under that assumption, but the exported denester's unconditional equality language does not state that its result has acquired an assumption-dependent domain. Once again, the whole-expression algebraic-number check is false for the symbolic host, so it does not guard this transformation.[1]

A useful regression embeds both a symbolic and a numerical radical:

```
Block[{$Assumptions = x > 0},
  RadicalDenest`Strad[Sqrt[x^2] + Sqrt[5 + 2 Sqrt[6]]]
]
```

The proposed policy is not to simplify the symbolic host at all. Numerical work is performed under `$Assumptions = True`, while each numerical replacement is proved equal locally. Substituting $x = -2$ into both input and output is a good regression for accidental leakage; it is not the general proof, which is Proposition 2.1.

An alternative API could accept an explicit `Assumptions` option and return a conditional result. That would be a different contract and would need corresponding domain-aware tests. It should not arise implicitly from the global state of the caller's notebook.

5 Further correctness and engineering findings

5.1 F08: Certification is conservative, but its failure states and timing are blurred

Location: `CertifiedEqualQ`, `ExactAlgebraicQ`. **Classification:** conservative false-negative risk and resource-control defect.

The baseline's numerical prefilter computes `N[a-b,30]` and rejects when the displayed magnitude exceeds 10^{-20}. This cannot directly cause acceptance of an incorrect identity, because acceptance still requires an exact zero test. It can, however, reject an identity before that exact test, and its fixed tolerance is not a certified error enclosure. Cancellation, coefficient height, and insufficient working precision matter; a small printed residual and a rigorously bounded residual are different objects. The recognition predicates and numerical approximation also execute outside the two explicit `TimeConstrained` expressions. Thus the helper is not globally bounded by its nominal certification allocation.[1]

Another subtlety is the expression `RootReduce[d] === 0`. It evaluates to `False` not only for a successfully

reduced nonzero algebraic number, but also when `RootReduce` remains unevaluated. The next statement immediately returns on `False`; consequently the `PossibleZeroQ` fallback is reached for a failed or timed-out first attempt, not for every undecided reduction. When reached, it can consume another complete certification allocation.

It would be inaccurate to diagnose `PossibleZeroQ[...,Method->"ExactAlgebraics"]` merely by the word "Possible" in the function name. Wolfram explicitly documents an exact guarantee for explicit algebraic numbers with this method. The concern here is recognition, control flow, and reporting, not an unsupported claim that this documented mode is probabilistic.[11]

The replacement removes the numerical prefilter. Its acceptance operation requires a bounded exact reduction of $a - b$ to the literal integer zero; unsupported forms, exhausted budgets, messages, and unevaluated reductions are conservative rejection states. Statistics distinguish certificates proved, rejected, and undecided. This is intentionally simpler than a sophisticated numerical filter. A later certified interval filter can reject when two outward-rounded enclosures are disjoint; overlapping enclosures should trigger refinement or exact algebra, never unconditional acceptance.

5.2 F09: A valid aggregate power identity does not license termwise redistribution

Location: `powerMergeSafeQ`, `combine`, `Factorc`. **Classification:** proof gap in a transformation guard; reachability not established.

Two different issues must be kept apart. The baseline's nested-power guard is substantially reasonable. For a nonzero complex z with principal argument in $(-\pi, \pi]$, a real inner exponent b satisfying $-1 < b \leq 1$ does not wrap $b \operatorname{Arg} z$ across the principal cut. Hence $(z^b)^c = z^{bc}$ in that guarded case, and integer outer exponents also justify the merge. This guard is not itself the counterexample. Zero bases and undefined powers must, as usual, be excluded by the domain.[1]

The extraction guard in `combine` addresses an aggregate factorization

$$(FS)^p = F^p S^p.$$

A positive real F, or a positive real S, is sufficient for this equality under principal powers; integer p also suffices where the expressions are defined. However, the implementation returns separately powered factor-list entries. From $F = \prod_j b_j^{e_j}$, it constructs terms resembling $\prod_j b_j^{e_j p}$. Positivity of the *product* F does not justify that further distribution.

Example 5.1 (Aggregate positivity is not enough)**.** Let $F = i(-1+i)^2 = 2$ and $p = 1/2$. The correctly aggregated result is $\sqrt{F} = \sqrt{2}$, whereas the termwise result is

$$i^{1/2}(-1+i) = -\sqrt{2}.$$

Indeed $i^{1/2} = (1+i)/\sqrt{2}$, and multiplying gives $-2/\sqrt{2}$. Thus a positive aggregate may have a termwise principal-power product with the wrong sign.

This example refutes the justification of the guard as a general algebraic statement. It is not presented as an executed input demonstrating that `FactorList` and all preceding normalizations necessarily produce exactly this internal factor-list state. Moreover, the baseline's final `Factorc` equality check protects exact numerical callers: a wrong candidate should be discarded rather than returned. The practical consequences can therefore be lost opportunities and wasted work, not necessarily a wrong default result.

The replacement avoids this recursive factor-list language altogether. Its `Factorc` is a bounded built-in factoring proposal, accepted only for supported exact numerical inputs and only after equality and cost checks.

That is a deliberate reduction in repertoire. A future custom factorizer should use an explicit expression-tree representation whose nodes distinguish a product, a sum, and a power, and attach a branch condition or certificate to every nontrivial extraction.

5.3 F10: Certification should survive polishing, not just precede it

Location: `tryMultiplier`, `polish`, final engine selection. **Classification:** trust-boundary and availability issue.

The baseline first filters raw possibilities by equality with the target. It then expands those possibilities into several polished representations and chooses the cheapest. The statement that all polish variants preserve value is an additional proof obligation; it is not inherited automatically from the certificate for the original candidate. The public rationalization failure in F06 illustrates why this distinction is operationally important.[1]

For the default exact-numerical path, the later whole-result certificate is a useful backstop. It prevents a bad final variant from being returned. But if an invalid, very cheap variant displaces a valid alternative before that final check, the engine can ultimately return the original input and lose a denesting it had already found. A final gate therefore supplies safety without necessarily supplying good failure recovery.

The replacement keeps an immutable target and a best certified candidate. Each original or polished proposal passes through the same `choose` operation. A failed proposal cannot overwrite the best value. A successful local expression and its cost are published together only after their checks finish. The implementation currently offers ordinary simplification and factoring as polishing proposals; it does not claim to preserve every old rationalization or recursive-polish heuristic.

5.4 F11: The cost model is underspecified and is not compositional

Location: `RadicalCost`, `RadicalDepth`, wrapper selection. **Classification:** objective mismatch and optimization limitation.

The baseline usage string describes four components, while its implementation returns five: opaque algebraic-object count, rational-power depth, rational-power count, atomic count, and `LeafCount`. This is a documentation defect, not an arithmetic error. The comparison `Order[newCost,oldCost] === 1` is also not reversed: `Order` returns 1 when its first argument precedes its second in canonical order. With these equal-length integer tuples, the intended lexicographic comparison is appropriate.[1, 16]

There are more substantial modeling questions. A `Root` object is an algebraic-number representation, not a depth-zero radical solution; counting it before depth is a reasonable policy, but its defining function should not be traversed as if it were a radical formula for the selected number. The baseline does count radicals inside such representation payloads. Similarly, a phase written using an exponential should not obtain a spurious zero-radical score merely by avoiding the `Power[_,_Rational]` pattern. A cost function measures a specified grammar, not an arbitrary printing convention.

The replacement uses

$$C(E) = \big(\#\text{opaque algebraic objects},\ \operatorname{depth}(E),\ \#\text{rational-power nodes},\ \operatorname{LeafCount}(E),\ H(E)\big), \quad (2)$$

where H counts the bit lengths of integer and rational coefficients, including the rational real and imaginary components of complex atoms. `Root` and `AlgebraicNumber` payloads are opaque to radical-depth counting. The exact atom i is allowed as a Gaussian-rational constant. Consequently this is an explicit syntactic objective with i effectively available, not a theorem about minimum radical depth over $\mathbb{Q}$.

Even this clarified lexicographic cost is not compositional. Suppose one term in a sum already fixes the maximum radical depth. Denesting another term can lower its local depth while increasing its number of nodes or leaves; the whole sum's depth stays unchanged, so its overall cost can increase. For example, the local identity

$$\sqrt{2+\sqrt{3}} = \frac{\sqrt{2}+\sqrt{6}}{2}$$

can replace a deep but compact term by a shallower, larger one while a different term still controls the maximum depth. Local strict improvement does not imply strict improvement of every containing expression.

Accordingly the replacement performs a separate whole-pass cost check. This preserves the advertised non-increase policy but can reject an entire pass even when some subset of its local changes would have been useful. A globally optimal selection would require richer information: retain a small Pareto frontier of depth, node count, and size for each subtree, or perform a bounded beam search over combinations. The present all-or-nothing pass is simpler and conservative, not globally optimal.

5.5 F12: Search progress, deduplication, and representation need separate invariants

Location: multiplier history, stack, visited set, and degree-based scheduling. **Classification:** heuristic coverage and maintainability limitation.

The baseline's visited set is keyed by the written multiplier. Algebraically equal multipliers can therefore consume different trials. Its history uses minimal-polynomial degree to schedule progress, but degree is not radical depth: denesting changes representation without changing the algebraic degree of the target. A lower degree of an auxiliary root may be useful evidence about the multiplier, but is not a proof that the final representation is cheaper. The separate cost check is essential.[1, 4]

Likewise, the least common multiple of denominators at maximal syntactic depth is a candidate reduction exponent, not a theorem that powering an arbitrary sum or product by that integer decreases its depth. Expansions and cancellations determine what actually happens. A large least common multiple can produce a large binomial polynomial and a large roots-of-unity orbit even when the original expression is short.

The stack combines partially processed entries, deferred generator calls, history references, and degree data in positional lists. This creates substantial coupling between scheduling, data representation, and mathematical computation. Guarding every list access treats symptoms; a small state machine with explicit records is easier to reason about and test.

The replacement intentionally chooses a less elaborate search discipline: finite FIFO admission, exact canonical keys, an immutable target, and a single insertion gate. It bounds root order and polynomial degree separately, and records degrees as diagnostics rather than proof of denesting. It does not retain the original scheduling priority, so it may explore a different subset of the search space under the same nominal trial count. That is a coverage tradeoff, not a demonstrated speed improvement.

5.6 F13: Traversal and algebraic representations need an explicit public contract

Location: `ToRadicals`, marker passes, list dispatch, inexact-input guard. **Classification:** restricted coverage and interface ambiguity.

The baseline converts `Root` and `AlgebraicNumber` objects before the main search, but resets the entire conversion if any such object remains. Thus one nonconvertible object can discard a successful conversion of another. `ToRadicals` is a representation proposal, not a guarantee that every algebraic number will become a useful radical formula; partial conversion should be judged locally.[1, 17]

The marker design also makes "all levels" depend on repeated rewrite traversal and on the order in which marker-containing problems are resolved. The resolution loop has an explicit iteration cap; this review does not claim that this particular loop has no limit. The criticism is that nesting, extraction, grouping, and execution are intermixed, which makes its semantic boundary harder to inspect than an ordinary recursive walk.

The new walker descends only through `Plus`, `Times`, `Power`, and `List`. Unknown heads, held syntax, associations, and algebraic representation payloads are deliberately opaque. This avoids pretending that replacing equal mathematical values preserves arbitrary Wolfram programs: a user-defined head can inspect syntax or have side effects. Ordinary argument evaluation still precedes the call, so the package is not a general held-AST editor.

The baseline returns an entire host containing inexact numbers without attempting its exact islands. The replacement can visit exact arithmetic components while leaving inexact atoms untouched, but it does not promise preservation of every floating-point evaluation path. Both policies should be stated rather than inferred from an implementation detail. Expressions with numerical effects or user-defined evaluation rules belong outside the denester's mathematical-host contract.

5.7 F14: Two earlier improvement proposals need mathematical correction

Location: the earlier review's `sec_future.tex`, not the current direct-surd routine. **Classification:** exact mathematical counterexamples.

The indirect criterion has a sign. The earlier improvement plan states a fourth-root criterion using $c(a^2 - b^2c)$. In the real quadratic-over-quadratic setting, the genuine indirect criterion is that

$$-c(a^2 - b^2c)$$

be a rational square, with the accompanying positivity conditions. The unified mathematical report already gives the correct sign. For $\sqrt{4 + 3\sqrt{2}}$, the norm is $D = -2$, and $-cD = 4$ is a square whereas $cD = -4$ is not a rational square. The explicit fourth-root answer is $2^{1/4}(1 + \sqrt{2})$. This is a correction to the proposed extension, not an allegation that the baseline's implemented direct formula uses the wrong norm.[3, 4, 9]

Norm divisors alone are not a sufficient multiplier family. The earlier plan also proposes a norm-derived finite family as if its sufficiency followed generally from a Kummer-type description. Norm information alone loses essential multiplicative information. Consider

$$K = \mathbb{Q}(\sqrt{3}), \qquad \rho = 2 + \sqrt{3}, \qquad N_{K/\mathbb{Q}}(\rho) = 1.$$

If $\sqrt{\rho} = A + B\sqrt{3}$ with rational A, B, then $A^2 + 3B^2 = 2$ and $2AB = 1$, which force $A^2 \in \{1/2, 3/2\}$. Neither is a rational square, so $\sqrt{\rho} \notin K$. Since K is real, $K(i) \cap \mathbb{R} = K$; adjoining the square roots of the rational norm divisors ± 1 therefore still does not contain $\sqrt{\rho}$.

Nevertheless multiplier $m = 2$ works:

$$\sqrt{2\rho} = 1 + \sqrt{3} \in K, \qquad \sqrt{\rho} = \frac{1 + \sqrt{3}}{\sqrt{2}}.$$

Thus the norm-one condition does not reduce the problem to the norm's rational prime divisors. A correct field-theoretic replacement needs multiplicative classes, units, and the stated base-field hypotheses, not just the integer factorization of a rational norm.[3]

This counterexample does not invalidate the current discriminant heuristic; that heuristic uses different data and makes no valid completeness claim in the first place. It invalidates the stronger proposed sufficiency argument. A future exact algorithm should formulate its target subgroup of $K^\times/(K^\times)^q$ explicitly before selecting representatives.

6 Exact low-degree constructions for a better front end

General algebraic-number operations are expensive enough that cheap structure-specific methods deserve priority. The formulas below are not an unrelated identity catalog: they are the mathematical basis of the front end in `StradImproved.wl`. Every reconstructed candidate still passes the same exact acceptance gate. The proofs explain why the recipes work and where they stop.

6.1 Direct quadratic denesting

Let $a, b, c \in \mathbb{Q}$, with $c > 0$ nonsquare and $b \neq 0$, and write

$$\rho = a + b\sqrt{c} > 0, \qquad D = a^2 - b^2 c.$$

The classical direct test uses the quadratic-field norm D. In the real setting, the structural literature shows that the rational-square condition characterizes denesting into real square roots of rationals. The following elementary proof establishes the explicit construction used here; the broader necessity statement is attributed to BFHT rather than inferred merely from a two-term ansatz.[6, 9]

Proposition 6.1. *If $D = s^2$ for $s \in \mathbb{Q}_{\geq 0}$, then*

$$\sqrt{a + b\sqrt{c}} = \sqrt{\frac{a+s}{2}} + \operatorname{sgn}(b)\sqrt{\frac{a-s}{2}}. \tag{3}$$

Both square-root arguments on the right are nonnegative, and the right side is positive.

Proof. Since $D \geq 0$, we have $|a| \geq |b|\sqrt{c}$. If $a \leq 0$, then $a + b\sqrt{c} \leq 0$, contradicting the hypothesis; hence $a > 0$. Also $0 \leq s < a$ because $a^2 - s^2 = b^2 c > 0$. Set $u = (a+s)/2$ and $v = (a-s)/2$. Then $u, v > 0$, $u + v = a$, and $uv = b^2 c/4$. Squaring $\sqrt{u} + \operatorname{sgn}(b)\sqrt{v}$ gives $a + b\sqrt{c}$. For $b > 0$ the expression is positive. For $b < 0$, s cannot be zero: otherwise $c = (a/b)^2$ would be a rational square. Thus $u > v$ and the difference is positive as well. The principal root is therefore selected. □

The degenerate cases $b = 0$, rational-square c, or $\rho = 0$ should be normalized separately. The implementation's pattern recognizer requires a single positive rational square root after expansion and rational coefficients. It is therefore intentionally narrower than a general algorithm for quadratic fields written in an arbitrary algebraic basis.

6.2 Indirect quadratic denesting and the correct sign

Proposition 6.2. *Suppose $D < 0$ and*

$$e = \sqrt{b^2 - a^2/c} \in \mathbb{Q}.$$

Then $b > 0$, and

$$\sqrt{a + b\sqrt{c}} = c^{1/4}\left(\sqrt{\frac{b+e}{2}} + \operatorname{sgn}(a)\sqrt{\frac{b-e}{2}}\right). \tag{4}$$

The condition on e is equivalent to $-cD$ being a rational square.

Proof. The inequality $D < 0$ gives $|a| < |b|\sqrt{c}$. If $b < 0$, then $a + b\sqrt{c} < 0$, contrary to $\rho > 0$; hence $b > 0$. We have $0 < e \le b$, so both square-root arguments are nonnegative. Their product is $a^2/(4c)$. Consequently the square of the parenthesized expression is

$$b + \operatorname{sgn}(a)\sqrt{b^2 - e^2} = b + \frac{a}{\sqrt{c}}.$$

Multiplication by $\sqrt{c}$ proves the squared identity. The parenthesized expression is positive: for $a \ge 0$ this is immediate, and for $a < 0$ the first square root is strictly larger than the second. Finally, $-cD = c^2e^2$, and c is nonzero rational, giving the equivalence. □

The examples

$$\sqrt{4 + 3\sqrt{2}} = 2^{1/4}(1 + \sqrt{2}), \qquad \sqrt{-4 + 3\sqrt{2}} = 2^{1/4}(\sqrt{2} - 1)$$

exercise both signs of a. For $a = 0$, the term multiplied by $\operatorname{sgn}(a)$ vanishes and the same formula has the expected interpretation.

The direct and indirect alternatives form the relevant local real classification in the cited literature. They do not establish that square and fourth roots suffice for arbitrary deeper radical expressions or unrestricted complex denesting. The implementation uses their sufficient formulas, not an unrestricted impossibility oracle.[6, 9]

6.3 Negative radicands and composite outer indices

For an exact negative real ρ, principal square-root semantics gives

$$\rho^{1/2} = i(-\rho)^{1/2}.$$

The package first tests the sign exactly, applies the positive-radicand formula when available, and restores the factor i. No numerical sign threshold chooses the branch.

Composite indices offer another inexpensive opportunity. If q is even, successive principal extraction of a square root and a $(q/2)$th root agrees with the principal qth root. For $\rho \neq 0$, write $\rho = Re^{i\theta}$ with $-\pi < \theta \le \pi$. The first extraction has argument $\theta/2$; the next has argument θ/q, without crossing a principal cut. The zero case is immediate. This justifies the recursive recipe, but intermediate expressions still require care with their representation.

For example,

$$(49 + 20\sqrt{6})^{1/4} = \sqrt{5 + 2\sqrt{6}} = \sqrt{2} + \sqrt{3}. \tag{5}$$

The middle representation need not be a strict improvement under every cost convention. Requiring every internal construction step to improve the global objective would therefore be unnecessarily restrictive. The package permits cost-neutral intermediate values inside a single formula recipe, certifies the resulting root, and applies the strict cost gate only when proposing a replacement to the surrounding expression.

For a general rational exponent p/q in lowest terms, the recipe proposes the integer power p of the reconstructed principal qth root. Integer outer powering gives the correct value; the exact final gate remains in force. This also handles negative rational exponents when the base is nonzero.

6.4 A cubic root that remains in a real quadratic field

Let $c > 0$ be nonsquare rational, and let $a, b \in \mathbb{Q}$, $b \neq 0$. We seek a *real* cube root of $a + b\sqrt{c}$ of the restricted form $A + B\sqrt{c}$ with rational A, B. This is a field-membership problem, not unrestricted cubic denesting. The

report gives the following trace–norm formulation, and the proof below supplies the reconstruction identities needed by the code.[4]

Theorem 6.3. *Such a root exists if and only if there are rational numbers n, T satisfying*

$$n^3 = a^2 - b^2c, \qquad T^3 - 3nT - 2a = 0. \tag{6}$$

In that case $T^2 - n \neq 0$ and

$$A = \frac{T}{2}, \qquad B = \frac{b}{T^2 - n}. \tag{7}$$

Proof. Assume first that $(A + B\sqrt{c})^3 = a + b\sqrt{c}$. Taking the quadratic norm yields $(A^2 - cB^2)^3 = a^2 - b^2c$. Define $n = A^2 - cB^2$ and $T = 2A$. Comparing the rational parts of the cubic gives

$$T^3 - 3nT = 8A^3 - 6A(A^2 - cB^2) = 2a.$$

The irrational coefficient gives

$$b = B(3A^2 + cB^2) = B(4A^2 - n) = B(T^2 - n).$$

Since $b \neq 0$, the denominator is nonzero, proving necessity and the reconstruction.

Conversely, suppose (6) holds. Squaring $T(T^2 - 3n) = 2a$ and rearranging gives

$$(T^2 - 4n)(T^2 - n)^2 = 4(a^2 - n^3) = 4b^2c. \tag{8}$$

Its right side is positive, so $T^2 - n \neq 0$. With A, B from (7), division of (8) yields

$$cB^2 = \frac{T^2 - 4n}{4} = A^2 - n.$$

Hence the rational cubic coefficient is

$$A^3 + 3AcB^2 = 4A^3 - 3nA = \frac{T^3 - 3nT}{2} = a,$$

and the irrational coefficient is

$$3A^2B + cB^3 = B(4A^2 - n) = b.$$

Thus the cube is exactly $a + b\sqrt{c}$. The reconstructed expression is real, and a real number has a unique real cube root. □

The rational n, when present, is found by an exact real cube-root test on the rational norm. Rational T values are found as linear factors of a degree-three rational polynomial, not by scanning integer divisors of a potentially large coefficient. The discriminant of this cubic is $-108b^2c < 0$, so it has exactly one real root; this is consistent with uniqueness in the stated nondegenerate real setting.

The principal complex value still has to be selected. For example,

$$(1 - \sqrt{2})^3 = 7 - 5\sqrt{2} < 0,$$

but `(7-5 Sqrt[2])^(1/3)` is not $1 - \sqrt{2}$. The implementation generates the three candidates obtained by multiplying the real reconstructed root by cube roots of unity and certifies them against the original principal `Power`. Thus a real-algebraic construction can be used safely without silently changing the API's branch convention.

6.5 A Honsbeek-based square-root-of-cube-roots recipe

Let A, B be real numbers with $A^3 = \alpha$, $B^3 = \beta$, where $\alpha, \beta \in \mathbb{Q}^*$, and suppose $A + B > 0$. Set $q = \beta/\alpha$. A useful candidate-generating equation from the report's discussion of Honsbeek is

$$F_q(s) = s^4 + 4s^3 + 8qs - 4q = 0, \qquad s \in \mathbb{Q}. \tag{9}$$

The structural classification in Honsbeek's work has its own radical-closure and nondegeneracy hypotheses. The replacement only needs the following directly proved certificate, and implements the real positive-denominator branch of the construction.[4]

Proposition 6.4. *If (9) holds and $\beta - s^3\alpha > 0$, then the two candidates*

$$\pm\frac{-s^2A^2/2 + sAB + B^2}{\sqrt{\beta - s^3\alpha}} \tag{10}$$

have square $A + B$. Exactly the nonnegative one is the principal square root.

Proof. The following polynomial identity holds for all u, s:

$$\left(-\frac{s^2}{2} + su + u^2\right)^2 - (u^3 - s^3)(1 + u) = \frac{s^4 + 4s^3 + 8u^3s - 4u^3}{4}. \tag{11}$$

This follows by expanding both sides and collecting coefficients. Set $u = B/A$, so that $u^3 = q$, and multiply by A^4. The right side is zero, and the left side becomes

$$\left(-\frac{s^2A^2}{2} + sAB + B^2\right)^2 - (\beta - s^3\alpha)(A + B).$$

Dividing by the positive nonzero denominator proves the square identity. Since $A + B > 0$, the numerator is nonzero and one sign of the quotient is positive. □

As a benchmark, take $A = \sqrt[3]{5}$ and $B = -\sqrt[3]{4}$. Then $q = -4/5$, $s = -2$ solves the quartic, and $\beta - s^3\alpha = 36$. The construction gives the familiar denesting

$$\sqrt{\sqrt[3]{5} - \sqrt[3]{4}} = \frac{\sqrt[3]{2} + \sqrt[3]{20} - \sqrt[3]{25}}{3}. \tag{12}$$

Reversing the order of the two real cube-root terms gives $q = -5/4$ and $s = 1$, with denominator square 9. A correct implementation must not make denestability depend on the order of summands. The independent checks verify both parameterizations and the underlying polynomial identity.

The current recognizer requires exactly two real summands whose cubes reduce to nonzero rationals. It does not implement the entire complex classification, nor every degenerate rational-cube-ratio simplification. These are clearly bounded extension opportunities rather than cases to be forced into an inappropriate formula.

6.6 Denominator inversion by a polynomial certificate

The rationalization helper need not divide a polynomial by $x - d$ in a coefficient domain inferred from the expression. A more explicit construction is available.

Proposition 6.5. *Let $d \neq 0$ be algebraic, and let*

$$p(x) = a_0 + a_1 x + \cdots + a_n x^n \in \mathbb{Q}[x], \qquad p(d) = 0,$$

with $a_0 \neq 0$. Then

$$\frac{1}{d} = -\frac{a_1 + a_2 d + \cdots + a_n d^{n-1}}{a_0}. \tag{13}$$

Proof. The equality $p(d) = 0$ is equivalent to

$$d(a_1 + a_2 d + \cdots + a_n d^{n-1}) = -a_0.$$

Division by the two nonzero quantities gives the result. □

The numerator is evaluated in Horner form, requiring $n - 1$ multiplications rather than separately building every power. For a minimal polynomial of a nonzero algebraic number, a zero constant term would contradict irreducibility, so that situation is a failure of the expected contract and is rejected. The code verifies rational coefficients, degree, and nonzero constant term, constructs the proposed inverse, and certifies $d \cdot \text{inverse} = 1$ before replacing a denominator. This local certificate also permits a symbolic numerator without asking an algebraic-number oracle to prove a symbolic identity.

For $d = 1 + \sqrt{2}$, the polynomial $x^2 - 2x - 1$ gives $d^{-1} = d - 2 = \sqrt{2} - 1$. This operation can enlarge the expression, so the exported rationalization helper does not promise to lower the denesting cost. Its contract is a certified rational denominator when the construction succeeds, with unchanged-input fallback.

7 The proposed implementation

7.1 Package boundary and entry points

The replacement is delivered as `StradImproved.wl` in the separate context `RadicalDenestImproved‘`. It neither loads nor overwrites the baseline package. Fully qualified names are recommended when comparing implementations:

```
Get["/full/path/to/StradImproved.wl"];
RadicalDenestImproved`Strad[Sqrt[4 + 3 Sqrt[2]]]
RadicalDenestImproved`Strad[(49 + 20 Sqrt[6])^(1/4), True]
RadicalDenestImproved`DenestReport[
  Sqrt[1 + Sqrt[2]], "MaxTrials" -> 30,
  "TimeBudget" -> 10, "Trace" -> True]
```

The familiar expression-returning heads remain available: `Strad`, `DenestRadicals`, and `DenestCore`. The new `DenestReport` returns the expression together with status, limits reached, effective options, operation statistics, and a bounded trace. `DenestCore` treats one numerical target; it is not an alternative symbolic-host traversal.

Every entry point resolves its own current `Options` and any supplied rules before starting work. For example, `SetOptions[RadicalDenestImproved‘Strad, ...]` affects that head; it does not silently alter all the other heads. Legacy positional `True`/`False` calls become an explicit `"AllLevels"` rule. Unknown options and invalid finite limits produce a structured `Failure`, not a misleading unchanged mathematical result.

7.2 One acceptance operation

The most important design reduction is that neither a fast path, an algebraic solver, nor a user callback can publish its own answer. They produce proposals. The following abbreviated excerpt shows the invariant:

```
choose[target_, best_, candidate_, method_String] := Module[{},
  If[candidate === $Failed || candidate === $Aborted ||
      ! algebraicFormQ[candidate] || ! smallQ[candidate] ||
      ! cheaperQ[candidate, best], Return[best]];
  If[certified[candidate, target],
    bump["CandidatesAccepted"];
    note[method, <|"BeforeCost" -> RadicalCost[best],
      "AfterCost" -> RadicalCost[candidate]|>]; candidate,
    best]
];
```

The target does not change as the best candidate changes. An unsuccessful certificate does not mean that the candidate is mathematically different; it means that this operation lacks an acceptable certificate within its current domain and budget. That distinction is reflected in the statistics rather than hidden inside a Boolean.

The explicit numerical grammar admits rational and Gaussian-rational constants, arithmetic combinations, rational powers, and exact `Root`/`AlgebraicNumber` values. The syntax check is followed by bounded exact numerical/algebraic recognition. Arbitrary expressions such as unevaluated exponentials, trigonometric constants, or symbolic calls do not obtain algebraic status merely because a numerical approximation is available. A kernel simplification that already turns one into a supported exact form before entry is a different matter.

Roots of unity generated internally are represented by rational powers of -1. This keeps their representation inside the grammar and makes their rational-power nodes visible to the cost. The selected branch is determined by equality with the target, not by interpreting the appearance of this representation.

7.3 Marker-free traversal and whole-pass commits

The walker has two modes. With `"AllLevels" -> False`, an exact fractional-power node is offered as one target without first rewriting its interior; arithmetic components outside that node are still visited. With `True`, children are visited first, and successful whole passes can be repeated up to `"MaxPasses"`. Lists share one session rather than invoking a fresh top-level search for every element.

Each numerical replacement has its own local certificate. At the end of a pass, the candidate must also be cheaper than the last committed whole expression. If the original whole input is an explicit algebraic number, the code additionally certifies the entire result against that original input. Symbolic mathematical hosts instead use the local congruence argument in Proposition 2.1.

A snapshot consists of the committed expression and its cost. It is replaced with one assignment after the next expression has passed all applicable checks and its cost has been computed. A time or memory interruption returns the last published snapshot, not a partially transformed marker tree. This is an implementation invariant to be checked in native testing, not a claim that arbitrary Wolfram evaluation can be made transactional: user callbacks can still cause external side effects, and ordinary argument evaluation happens before the session begins.

7.4 A finite queue with one admission gate

For each numerical search target, `multiplierSearch` has a queue, a cursor, an exact-key visited association, an admission count, and a proposal count. All seed and generated candidates enter through a single closure. The closure validates size and exactness, canonicalizes the multiplier, rejects zero and duplicate keys, and only then admits it.

Let C be `"MultiplierCap"`. The implemented structural bounds are

$$\#\text{admissions} \le C, \qquad \#\text{proposal attempts} \le 4C. \tag{14}$$

A trial advances the cursor and increments the shared session trial counter. No rejected candidate bypasses the admission gate, and no generator constructs an unrestricted list of divisor combinations before applying a cap. The finite queue can be materialized safely because its length is bounded; it is the combinatorial *generation* that is incremental.

Automatic seeds include small rational multipliers and complements of visible rational powers. A supplied list replaces that initial seed list, but does not bypass the cap. Discriminant-derived candidates use a bounded set of primes and a bounded prefix of mixed-radix exponent vectors. This is still a heuristic family. Its small prefixes deliberately favor certain products and cannot establish that omitted products are useless.

The generic trial computes

$$\theta = \mathrm{RootReduce}(m\rho), \qquad p = \mathrm{MinPoly}_{\mathbb{Q}}(\theta^{1/q}),$$

then forms the polynomial GCD over the explicitly specified field $\mathbb{Q}(\theta)$. A linear factor is solved by its coefficients; higher proper-degree factors go through a bounded `Solve`, with optional `ToRadicals`. The root-of-unity orbit is enumerated one candidate at a time. No full `Outer` array of roots and phases is needed.

An exact canonical key prevents repeated trials on algebraically equal multiplier values. It does not identify all multipliers that are equivalent modulo qth powers in a field; exploiting that coarser equivalence requires a mathematical normalization that is outside the present implementation.

7.5 Resource controls and what they actually bound

Option	Default	Meaning
`"TimeBudget"`	120 seconds	Enclosing timed computation for one public call; shared by list elements and passes.
`"MaxTrials"`	400	Shared count of general multiplier trials; formula fast paths do not consume trials.
`"OperationTime"`	5 seconds	Per-operation allocation, additionally clipped to remaining session time.
`"CertifyTime"`	20 seconds	Per recognition/certification operation, also clipped to remaining time.
`"MemoryBudget"`	268435456 bytes	Additional allocation limit of the enclosing `MemoryConstrained` computation.
`"MultiplierCap"`	1000	Total admissions for one numerical search target, including all batches.
`"MaxRootOrder"`	32	Maximum root order used by formula recursion and the general search.

Option	Default	Meaning
`"MaxPolynomialDegree"`	64	Maximum accepted degree of an algebraic search polynomial.
`"MaxLeafCount"`	20000	Syntactic size guard on inputs and candidates checked by the engine.
`"MaxIntegerBits"`	65536	Total integer/rational coefficient-bit cost, not a per-coefficient bound.
`"MaxPasses"`	4	Maximum number of whole-expression passes in all-level mode.
`"PrimeLimit"`, `"BatchSize"`	8, 32	Primes retained per discriminant and proposal attempts per generated batch.
`"MaxTraceEntries"`	128	Maximum number of stored trace records when tracing is enabled.

An important distinction is between an allocation budget and process memory. Wolfram documents `MemoryConstrained` in terms of additional memory used during the computation; it does not impose a total resident-set-size limit on the entire kernel process. Similarly, `TimeConstrained` is a kernel control mechanism, not a hard real-time guarantee or an operating-system sandbox. Interrupt-protected operations can complicate enforcement. These limits are useful containment mechanisms, but production services requiring isolation should run work in separate processes with external limits.[15, 14]

Argument evaluation, option resolution, and final small report assembly are outside the timed body. A polynomial-degree check after `MinimalPolynomial` cannot prevent all work involved in discovering that the degree is too large; the enclosing operation and allocation limits are the corresponding protection. A user callback is opaque work: its own nested public calls may start nested sessions, while the outer time/memory boundary still surrounds the callback. The engine's trial counter does not measure arbitrary work or hidden trials inside user code.

A zero trial limit disables multiplier trials, not all simplification. A zero time budget returns the input without computing the initial cost, so cost metadata is `Missing["NotComputed"]`. A zero admission cap disables the multiplier queue. These behaviors are explicit and tested in the supplied native suite rather than left to incidental loop semantics.

7.6 Status and diagnostic interpretation

The report distinguishes `"Improved"` from `"Unchanged"`, with a `"WithLimits"` suffix when a limit has been recorded. It also includes effective options, elapsed time, initial and final cost, bounded trace, and operation/certification counts. The result contains `"CompletenessClaim" -> False` explicitly.

A limit flag means that a threshold was reached or an operation exhausted its allocation. It is not evidence that there was necessarily a useful unexamined candidate; for example, reaching the admission count exactly can set the flag even when no further beneficial multiplier exists. Conversely, `"Unchanged"` without a limit flag means only that these enabled methods found no acceptable improvement. It is not a certificate of non-denestability.

The current trace logs event names, costs, trial degrees, and root orders, not a portable formal proof object. Exact equality was established through the trusted algebraic-number implementation when a candidate was accepted. A durable independently checkable certificate would additionally record defining polynomials, field maps, and isolating data; that is an important future improvement.

7.7 Compatibility changes and remaining limitations

This is a conservative redesign, not a promise of identical output or a superset of the old search coverage. It replaces the custom factorizer, changes the traversal domain and search order, bounds root orders and polynomial degrees, and does not implement a general field-theoretic minimum-depth algorithm. Formula support is intentionally shape-specific. Higher odd root indices beyond the cubic fast path normally rely on the general heuristic engine.

The FIFO search may be less effective than a tuned priority scheduler on some examples. Canonical multiplier keys and repeated certificates can themselves be expensive. Global cost rejection may discard a useful subset of local changes. Early stopping once a radical-only depth-one candidate is found does not minimize every later component of the cost. Fixed resource defaults are engineering choices, not benchmark-derived recommendations. None of these tradeoffs is disguised as a correctness theorem or a measured speedup.

8 Validation and reproducibility

8.1 What was executed

The supplied `verification/verify.py` ran successfully with Python 3.13.5 and SymPy 1.14.0. It recorded 1,960 passing assertions: 1,956 exact mathematical or finite-model checks and four limited lexical checks of Wolfram files. The source hashes, detailed group counts, exact multiplier examples, and cap counterexamples are in `verification/results.json`; the human-readable execution output is in `verification/transcript.txt`.

Check group	Assertions
Direct quadratic construction	450
Negative-radicand branch construction	450
Indirect fourth-root construction	300
Wrong-sign indirect-criterion counterexample	1
Cubic trace–norm reconstruction	270
Honsbeek polynomial identity	1
Honsbeek rational parameter constructions	68
Reversed-order Honsbeek examples	1
Ramanujan square certificate	1
Horner inverse polynomial identities	40
Minimal-polynomial/GCD trial examples	2
Recovery of original-target roots-of-unity branches	4
Norm-only multiplier-family counterexample	1
Aggregate-factor branch counterexample	1
Baseline cap counterexamples	3
Finite bounded-scheduler model	363
Wolfram delimiters, strings, and comments	4
Total	**1960**

The mathematical checks use exact integers, rationals, symbolic polynomial identities, and quotient-ring

reductions. Branch conclusions use explicit real sign conditions. No floating-point near-equality is accepted as a proof. The finite scheduler model checks bounds and duplicate-handling behavior in its own explicitly coded state model; it is not a full translation of Wolfram evaluation semantics.

The lexical checks verify balanced delimiters with strings and nested comments handled separately. They do not parse Wolfram patterns, validate option scoping, or establish that a `SetDelayed` definition has the intended evaluation behavior. Their value is modest but real: they catch a class of construction errors without being inflated into a native test claim.

8.2 What was not executed

The native Wolfram evaluator was attempted and returned an MCP HTTP 404. No local Wolfram kernel was available. Accordingly, the accompanying 37 MUnit tests are *supplied but unrun*. The baseline diagnostic script is also unrun. No timing comparison, allocation benchmark, native parser success, or runtime success rate is asserted for the replacement.

The earlier repository review contains native results and timing claims from its own preparation. Those are historical evidence about that review's experiment, not a transcript produced in this audit. This article does not transfer those success claims to modified code or treat inherited measurements as a performance comparison.[2]

8.3 Native test categories

The suite covers exact-equality behavior, rejection of a wrong principal branch and a tiny nonzero residual, supported input forms, failed-polynomial sentinels, opaque `Root` costs, direct and indirect quadratic formulas, negative radicands, composite and negative exponents, cubic reconstruction, and the Honsbeek benchmark. Interface tests cover an untrusted solver in a symbolic host, ambient assumptions, unknown and held heads, symbolic factoring, denominator inversion, effective `SetOptions`, zero and malformed limits, forced-list admission caps, list-wide trial budgets, bounded diagnostics, and callback throws.

Several output regressions use `RootReduce[output-input] === 0` directly rather than calling the package's own equality predicate. This avoids making every integration test depend on the same wrapper under test, although both still trust the native algebraic kernel. The independent polynomial identities provide a different kind of cross-check for the mathematical recipes.

The runner first discovers the `TestReportObject` properties available in the installed kernel. Current documentation exposes `"Results"`, `"ReportSucceeded"`, and `"RuntimeFailures"`; a guarded legacy path supports the earlier count-property interface. An unsupported schema, empty or wrong-sized run, failed test, or reported runtime failure cannot silently produce a successful exit. This is important because a test runner is part of the validation chain, not incidental scaffolding.[18]

8.4 Reproduction commands

From the extracted directory:

```
python verification/verify.py
wolframscript -file tests/RunTests.wls
```

The first command reproduces the independently run checks. The second is the required next native validation step and writes `tests/native-results.json`. Use a fresh kernel so that old package definitions, altered

options, and global symbols cannot contaminate the result.

The optional baseline fetcher downloads only the pinned source and verifies its Git blob SHA-1 before saving it. Neither the replacement package nor its regression suite downloads or executes remote code automatically. To run the separate diagnostics:

```
python verification/fetch_baseline.py
wolframscript -file tests/BaselineProbes.wls \
  verification/StradFixed.baseline.wl
```

The diagnostic is deliberately not a native execution log embedded in advance. It prints observations for the user's installed kernel. Findings labeled as conditional failure paths or proof gaps retain those labels until the relevant behavior is actually reproduced.

8.5 An adoption gate, not just a unit-test count

Before replacing the old implementation in the repository, run the new suite and the repository's existing corpus in fresh kernels, retaining all native outputs and version metadata. For each changed result, check exact equality against the original input and the declared cost condition. Compare coverage under a shared wall-clock allocation rather than treating identical trial counts as identical computational effort. Include non-denestable or unrecognized examples, not only identities constructed to succeed.

A useful second stage generates answers first: construct sums of independent square roots or selected cubic-field elements, expand their powers, and present principal roots of those powers as inputs. Check signs or roots of unity explicitly when the constructed answer is not in the principal sector. Add adversarial coefficient heights, high root orders, duplicate multipliers in different representations, mixed symbolic hosts, and failing callbacks. Resource stress should measure actual elapsed time and process behavior, with interrupt-protected callbacks reported as outside the simple cooperative limit model.

Passing the independent checks does not replace this adoption gate. Their strongest conclusion is that the displayed algebra and finite counting arguments agree with exact independent computations, and that the supplied files pass the stated limited lexical checks.

9 Where to improve next

9.1 Use relative field structure before growing the search

For $\beta^q = \rho$, a natural first algebraic question is whether $x^q - \rho$ has a linear factor over an explicitly represented field K containing ρ. If it does, extracting that factor can avoid a search through many multipliers. More generally, low-degree factors can expose a smaller extension in which reconstruction is easier. An explicit coefficient field and selected embedding are essential: finding a conjugate root in an abstract isomorphic field is not enough to identify the desired complex value.

This is an implementation opportunity rather than a claim that field membership by itself minimizes radical depth. A field element may be printed in a deeply nested primitive-element basis; converting it into a low-depth expression is another task. The relevant literature distinguishes existence, field construction, and expression reconstruction, and the code should preserve the same separation.[6, 7, 4]

9.2 Represent square classes and dependencies explicitly

For a multiquadratic field generated by independent square classes $d_1, \dots, d_r$, the products

$$e_\varepsilon = \prod_{j=1}^{r} (\sqrt{d_j})^{\varepsilon_j}, \qquad \varepsilon \in \{0,1\}^r,$$

form a basis. A sign-changing automorphism indexed by $\eta \in \{0,1\}^r$ multiplies e_ε by $(-1)^{\eta\cdot\varepsilon}$. If $u = \sum_\varepsilon c_\varepsilon e_\varepsilon$, character orthogonality gives the exact projection

$$c_\varepsilon e_\varepsilon = 2^{-r} \sum_{\eta\in\{0,1\}^r} (-1)^{\eta\cdot\varepsilon} \sigma_\eta(u).$$

Indeed the inner sum over η is 2^r for the matching character and zero otherwise. This gives a principled basis for dependency-aware coefficient extraction and joint transformations of sums of radicals, rather than treating every written square root as an independent symbol.

The exponential basis size must remain visible. A sparse representation and lazy conjugation can make structured cases tractable, but do not magically make every r-generator problem polynomial in r. BFHT and the implementation discussion of Jeffrey–Rich are valuable guides precisely because successful local identities, recursive splits, and stopping rules must be designed together.[6, 8]

9.3 Replace norm-only intuition with multiplicative-class computations

The relevant search space for many multiplier questions is related to $K^\times/(K^\times)^q$, with additional roots-of-unity and base-field conditions. The norm map is only one projection of that information. Unit classes and elements with norm one can remain nontrivial, as F14 demonstrates.

A sound development path is to specify the allowed multiplier field, the allowed root indices, and the desired output field; derive a sufficient representative family under those hypotheses; and only then engineer its enumeration. Discriminant primes can remain a useful heuristic priority without being mistaken for a completeness certificate. The general algorithms in Landau's work motivate this distinction, while also warning that splitting-field and cyclotomic construction costs can dominate a short input formula.[7, 4]

9.4 Extend the trace–norm route to selected higher odd indices

If $u + v = T$ and $uv = n$, define

$$D_0(T,n) = 2, \qquad D_1(T,n) = T, \qquad D_{k+1}(T,n) = TD_k(T,n) - nD_{k-1}(T,n).$$

Multiplication by $u + v$ shows inductively that $D_k(T,n) = u^k + v^k$. Thus a proposed odd kth root remaining in a quadratic field must satisfy a rational norm condition and a trace equation $D_k(T,n) = 2a$. This generalizes the polynomial equation used in the cubic fast path.

It is a promising structured candidate generator, not an already implemented complete solver for every odd index. Rational trace roots must be reconstructed into the correct field element; denominator degeneracies and principal complex branches must be checked; uniqueness of the cubic trace root does not automatically persist in every higher-degree formulation. The repository report's Dickson-polynomial discussion is a useful starting point.[4]

9.5 Cache proofs and field representations without hiding failure

Repeated `RootReduce`, algebraic recognition, and primitive-field construction can dominate a bounded search. A bounded cache keyed by exact canonical data could reuse successful reductions, minimal polynomials, and field maps. A cached timeout should not be treated as a permanent mathematical fact: a later call with a larger budget or a different representation may succeed. Cache entries therefore need a status, resource context, and bounded storage policy.

A useful certificate format would contain a rational polynomial or quotient-ring identity and enough isolating information to identify the selected embedding. For example, a cubic recipe can record rational n, T and identity (8); a Honsbeek recipe can record rational s and (11). The remaining sign or complex-sector decision can be certified separately. This separates inexpensive algebraic checking from expensive candidate discovery and makes debugging less dependent on one opaque Boolean.

9.6 Optimize globally only after preserving local proofs

The next optimization layer should carry a small set of certified alternatives through each subtree. A Pareto frontier can retain a shorter deeper expression and a longer shallower one, postponing the decision until the containing expression's depth is known. A bounded beam can approximate this without claiming global minimum depth. Every frontier entry retains a certificate to the original local target; pruning affects coverage, not validity.

Search-priority tuning should follow reproducible measurements of certificate cost, field degree, coefficient height, allocation, and useful-success rate. A smaller minimal-polynomial degree can be one scheduling feature, but it should not be the objective or an unqualified success signal. The structured report added here supplies a starting point for such experiments; it does not substitute for them.

10 Conclusion

The first repair established the right central invariant in the numerical multiplier engine. The second audit finds that the invariant is not yet universal across the interface, and that several advertised resource and helper guarantees need stronger boundaries. The most urgent changes are local certification of all callback replacements, effective option resolution, real operation-level containment, a single capped admission path, and robust polynomial-failure validation.

The proposed replacement makes those boundaries explicit and adds useful exact front-end constructions. Its mathematical recipes and finite bounds have independent exact support; its Wolfram implementation still requires the supplied native regression run before adoption. It remains a bounded, branch-correct candidate search with a clearly stated syntax and objective, not a universal minimum-depth oracle. That is a stronger engineering foundation than either an aggressive uncertified simplifier or a safe engine whose wrapper quietly bypasses its proof obligations.

A Source-file map

File	Purpose
`StradImproved.wl`	Proposed package; separate namespace; complete implementation.
`tests/Regression.wlt`	Thirty-seven supplied native MUnit regressions.
`tests/RunTests.wls`	Fresh-kernel runner, discovered report properties, JSON summary, failing exit code on an unsuccessful run.
`tests/BaselineProbes.wls`	Unrun diagnostics for the pinned baseline, explicitly separated from new-code tests.
`verification/verify.py`	Executed exact mathematical checks, finite queue model, and lexical checks.
`verification/results.json`	Counts, examples, source hashes, and exact validation scope.
`verification/transcript.txt`	Output from the independent verification run.
`verification/fetch_baseline.py`	Optional pinned-source downloader with Git blob hash verification.
`PROVENANCE.json`	Source commit, target blob, and execution limitations.
`README.md`	Usage, compatibility changes, reproduction commands, and validation status.

B Reading the implementation by responsibility

Start with `resolveOptions`, `bounded`, and `runSession` to understand the public operational contract. Then read `algebraicFormQ`, `exactQ`, `certified`, and `choose`: they define the mathematical acceptance boundary. The cost functions and `walk` explain which subexpressions are visible and why local changes are followed by a whole-pass check.

The low-degree layer is self-contained: `quadraticParts`, `quadraticSquareCandidates`, `rationalPolynomialRoots`, `cubicQuadraticCandidates`, `honsbeekCandidates`, `fastRoot`, and `fastPower`. Their equations are proved in Section 6. The generic fallback is `multiplierSearch`; its nested `add` closure is the only multiplier admission point. Finally, `rationalizeRaw` and `factorSafe` provide the exported helper behavior without reintroducing the old custom factor-list representation.

The `Private‘` context is an encapsulation convention, not a security boundary. Native tests inspect a few private predicates for focused failure-path checks, but callers should normally use the public entry points and structured report rather than relying on internal names.

References

[1] V. Reshetnikov, *RadicalDenest repository: StradFixed.wl*, pinned commit `6d7739bda3478f0d9cd70ffb9f7c49ee2576bee7`, source blob `0625aca12ac28cbe16cb09489c22150aa294c1fc`. https://github.com/VladimirReshetnikov/RadicalDenest/blob/6d7739bda3478f0d9cd70ffb9f7c49ee2576bee7/src/corrected/StradFixed.wl. This is the reviewed implementation, not the replacement accompanying this article.

[2] *The corrected version*, in the repository's unified code review, `src/code-review/unified/sec_corrected.tex`, same pinned commit. https://github.com/VladimirReshetnikov/RadicalDenest/blob/6d7739bda3478f0d9cd70ffb9f7c49ee2576bee7/src/code-review/unified/sec_corrected.tex.

[3] *Future improvements*, in the repository's unified code review, `src/code-review/unified/sec_future.tex`, same pinned commit. https://github.com/VladimirReshetnikov/RadicalDenest/blob/6d7739bda3478f0d9cd70ffb9f7c49ee2576bee7/src/code-review/unified/sec_future.tex. The sign and norm-family qualifications in F14 concern this source.

[4] *Radical Denesting: A Unified Research Guide*, repository research report, `docs/report/radical_denesting_unified.tex`, same pinned commit. https://github.com/VladimirReshetnikov/RadicalDenest/tree/6d7739bda3478f0d9cd70ffb9f7c49ee2576bee7/docs/report. Used especially for direct/indirect denesting, trace–norm reconstruction, Honsbeek's quartic, and interpretation of the general algorithmic literature.

[5] *Corrected re-typesettings of six articles*, repository editorial inventory, `docs/literature/RETYPESET_ARTICLES.md`, same pinned commit. https://github.com/VladimirReshetnikov/RadicalDenest/blob/6d7739bda3478f0d9cd70ffb9f7c49ee2576bee7/docs/literature/RETYPESET_ARTICLES.md.

[6] A. Borodin, R. Fagin, J. E. Hopcroft, and M. Tompa, *Decreasing the nesting depth of expressions involving square roots*, Journal of Symbolic Computation **1** (1985), 169–188. Repository corrected edition inspected: `symb85.tex`, under the `bfht-Decreasing-the-nesting-depth-of-expressions-involving-square-root` literature record, `retypeset-2026`. https://github.com/VladimirReshetnikov/RadicalDenest/tree/6d7739bda3478f0d9cd70ffb9f7c49ee2576bee7/docs/literature/papers/bfht-Decreasing-the-nesting-depth-of-expressions-involving-square-root/retypeset-2026.

[7] S. Landau, *Simplification of nested radicals*, 1989 extended abstract; expanded paper, SIAM Journal on Computing **21** (1992), 85–110, https://doi.org/10.1137/0221009. The repository's corrected `landau1989.tex` was inspected, including its explicit roots-of-unity and base-field qualifications, under the `landau92-Simplification-of-nested-radicals/retypeset-2026` record.

[8] D. J. Jeffrey and A. D. Rich, *Simplifying square roots of square roots by denesting*, Chapter 4 in M. J. Wester (ed.), *Computer Algebra Systems: A Practical Guide*, Wiley, 1999. Repository corrected `denest.tex` inspected under `jeffrey-Simplifying-square-roots-of-square-roots-by-denesting/retypeset-2026`.

[9] E. Gkioulekas, *On the denesting of nested square roots*. Author-posted full text: https://faculty.utrgv.edu/eleftherios.gkioulekas/papers/submitted/denesting-square-roots.pdf. Also inspected: repository corrected `gkioulekas2017.tex` under `gkioulekas-On-the-denesting-of-nested-square-roots/retypeset-2026`. The indirect criterion was checked in the author PDF's displayed theorem.

[10] Wolfram Research, *RootReduce*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/RootReduce.html. Consulted 5 September 2026.

[11] Wolfram Research, *PossibleZeroQ*, especially the documented `Method -> "ExactAlgebraics"` guarantee for explicit algebraic numbers. https://reference.wolfram.com/language/ref/PossibleZeroQ.html. Consulted 5 September 2026.

[12] Wolfram Research, *PolynomialGCD*, including algebraic coefficient extensions. https://reference.wolfram.com/language/ref/PolynomialGCD.html. Consulted 5 September 2026.

[13] Wolfram Research, *OptionValue*, including effective option values and the explicit function/options/-name form. https://reference.wolfram.com/language/ref/OptionValue.html. Consulted 5 September 2026.

[14] Wolfram Research, *TimeConstrained*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/TimeConstrained.html. Consulted 5 September 2026.

[15] Wolfram Research, *MemoryConstrained*, including the additional-memory interpretation and interaction with interrupt protection. https://reference.wolfram.com/language/ref/MemoryConstrained.html. Consulted 5 September 2026.

[16] Wolfram Research, *Order*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Order.html. Consulted 5 September 2026.

[17] Wolfram Research, *ToRadicals*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/ToRadicals.html. Consulted 5 September 2026.

[18] Wolfram Research, *TestReportObject*, including current result, success, and runtime-failure properties. https://reference.wolfram.com/language/ref/TestReportObject.html. Consulted 5 September 2026. The runner discovers supported properties rather than assuming that one remembered report schema is universal.